Chapter 3 worked solutions

DS03-01 • Circular wraparound

Initially $(head, size) = (0, 0)$. A dash means that no live element occupies the slot. Any leftover bits in a removed slot are outside the abstract sequence. Enqueue uses slot $(head + size) \bmod 4$; dequeue sets $head = (head + 1) \bmod 4$.

Completed operation	head	size	Slot 0	Slot 1	Slot 2	Slot 3	Logical sequence
Initial	0	0	–	–	–	–	Empty
Enqueue A	0	1	A	–	–	–	A
Enqueue B	0	2	A	B	–	–	A,B
Enqueue C	0	3	A	B	C	–	A,B,C
Dequeue A	1	2	–	B	C	–	B,C
Dequeue B	2	1	–	–	C	–	C
Enqueue D	2	2	–	–	C	D	C,D
Enqueue E	2	3	E	–	C	D	C,D,E
Enqueue F	2	4	E	F	C	D	C,D,E,F
Dequeue C	3	3	E	F	–	D	D,E,F
Dequeue D	0	2	E	F	–	–	E,F
Dequeue E	1	1	–	F	–	–	F

Lin Cing https://github.com/xinhi6716/computer-science-self-study

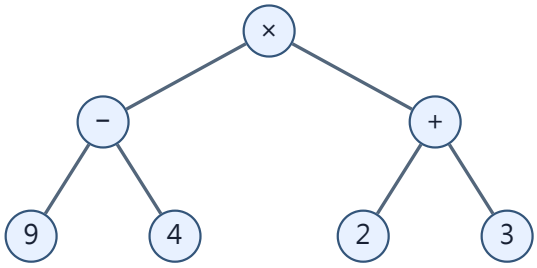
Full means size=4; empty means size=0. This representation does not use equality of head and a tail index to distinguish the two states.

DS03-02 • Expression conversion

The root is *. Its left subtree is − with leaves 9 and 4. Its right subtree is + with leaves 2 and 3.

Chapter 3 worked solutions (continued)

(9−4)×(2+3): left and right subtrees fix operand order



Prefix: * − 9 4 + 2 3

Postfix: 9 4 − 2 3 + *

Left: 9−4=5; right: 2+3=5

Final result: 5×5=25

Binary operator: pop right, then left

Prefix is `* - 9 4 + 2 3`. Postfix is `9 4 - 2 3 + *`. The postfix stack states are `[9]`, `[9,4]`, `[5]`, `[5,2]`, `[5,2,3]`, `[5,5]`, `[25]`. For `8 3 -`, the first pop 3 is the right operand and the second pop 8 is the left operand. The result is 5. Exchanging them gives −5 and changes the expression tree's meaning.

DS03-03 • The first DFS path

Use directed edges `s→a,a→b,b→c,c→t,s→t`, with a preceding t in s's neighbor order. DFS visits s,a,b,c,t and first reaches t in four edges. Its parents are a←s,b←a,c←b,t←c. BFS discovers both a and t while processing s, so t has parent s and distance 1. Continuing the whole BFS can give visit order s,a,t,b,c, with b←a and c←b. This counterexample concerns DFS neighbor order, not its correctness for reachability.

The complete BFS assigns a←s, t←s, b←a, and c←b. Distances are d(s)=0, d(a)=d(t)=1, d(b)=2, and d(c)=3. Assign a parent only on first discovery. Encountering the already discovered t from c does not change its parent to c.

DS03-04 • Syntax and safe evaluation

Reject `2 +` when + finds fewer than two operands. Reject `2 3` because two values remain at the end. Reject `8 0 /` when the right operand is zero. Also check integer-token parsing range, addition, subtraction, and multiplication overflow, and the unrepresentable result of the minimum signed integer divided by −1. If unary minus exists, token type must specify its arity. Stack size cannot determine the grammar.

Chapter 3 worked solutions (continued)

DS03-05 • End-of-input and grammar

After each token, stack / output are: 9 gives `empty / 9`; - gives `- / 9`; 4 gives `- / 9 4`; * gives `- * / 9 4`; 2 gives `- * / 9 4 2`; / gives `- / / 9 4 2 *`; 1 gives `- / / 9 4 2 * 1`; + gives `+ / 9 4 2 * 1 / -`; 3 gives `+ / 9 4 2 * 1 / - 3`; EOF gives `empty / 9 4 2 * 1 / - 3 +`. At +, both / and - must pop. Value is 4. An unmatched (in `(9-4` rejects at EOF. `9-4)` has no matching (at). `9-` still expects an operand at EOF. Right-associative postfix `2 3 2 ^ ^` yields 512; left-associative `2 3 ^ 2 ^` yields 64.

DS03-06 • Queue growth and the abstract value

Move C from old slot 2 to new 0, D from 3 to 1, E from 0 to 2, and F from 1 to 3. Commit head=0,size=4,capacity=8. Copying old slots and retaining head=2 makes logical index 2 access `(2+2) mod 8=4`, an unconstructed slot rather than E. Logical-order relocation costs Θ(n), allowing Θ(n) worst-case enqueue. Doubling only when full with constant-time moves gives a geometric total of Θ(n) over n enqueues from empty, hence Θ(1) amortized. Nonconstant value costs must be added. This differs from a fixed-capacity worst-case constant bound.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter 3 worked solutions (continued)

Verification code appendix: expression_reference.py

```
from fractions import Fraction
import json
import random
import re

PRECEDENCE = {'+':1, '-':1, '*':2, '/':2, '^':3}

def tokenize(expression):
    tokens = re.findall(r'\d+|[()+*/^--]', expression)
    if re.sub(r'\s+', '', expression).replace('−', '-') != ''.join(tokens).replace('−', '-'):
        raise ValueError('unknown token')
    return [token.replace('−', '-') for token in tokens]

def postfix(expression):
    output, stack, trace = [], [], []
    operand_expected = True
    for token in tokenize(expression):
        if token.isdigit():
            if not operand_expected:
                raise ValueError('missing operator')
            output.append(token)
            operand_expected = False
        elif token == '(':
            if not operand_expected:
                raise ValueError('missing operator before parenthesis')
            stack.append(token)
        elif token == ')':
            if operand_expected:
                raise ValueError('missing operand')
            while stack and stack[-1] != '(':
                output.append(stack.pop())
            if not stack:
                raise ValueError('unmatched right parenthesis')
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Chapter 3 worked solutions (continued)

```
            stack.pop()
            operand_expected = False
        else:
            if operand_expected:
                raise ValueError('binary operator needs left operand')
            while stack and stack[-1] != '(' and (
                PRECEDENCE[stack[-1]] > PRECEDENCE[token] or
                (PRECEDENCE[stack[-1]] == PRECEDENCE[token] and token != '^')):
                output.append(stack.pop())
            stack.append(token)
            operand_expected = True
        trace.append((token, ' '.join(stack) or 'Ø', ' '.join(output) or 'Ø'))
    if operand_expected:
        raise ValueError('empty expression or missing final operand')
    while stack:
        if stack[-1] == '(':
            raise ValueError('unmatched left parenthesis')
        output.append(stack.pop())
    trace.append(('EOF', 'Ø', ' '.join(output)))
    return output, trace

def apply(op, a, b):
    if op == '+': return a+b
    if op == '-': return a-b
    if op == '*': return a*b
    if op == '/': return a/b
    if b.denominator != 1 or abs(b) > 20:
        raise ValueError('exponent outside demonstration domain')
    return a**int(b)

def evaluate_postfix(tokens):
    stack = []
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Chapter 3 worked solutions (continued)

```
    for token in tokens:
        if token.isdigit():
            stack.append(Fraction(token))
        else:
            if len(stack) < 2: raise ValueError('operand underflow')
            b, a = stack.pop(), stack.pop()
            stack.append(apply(token,a,b))
    if len(stack) != 1: raise ValueError('extra operands')
    return stack[0]

def parse_reference(expression):
    tokens = tokenize(expression)
    position = 0
    def parse(minimum=0):
        nonlocal position
        token = tokens[position]
        position += 1
        if token == '(':
            value = parse()
            if tokens[position] != ')': raise ValueError('reference parenthesis')
            position += 1
        elif token.isdigit():
            value = Fraction(token)
        else:
            raise ValueError('reference operand')
        while position < len(tokens) and tokens[position] in PRECEDENCE and
PRECEDENCE[tokens[position]] >= minimum:
            op = tokens[position]
            position += 1
            rhs = parse(PRECEDENCE[op] + (op != '^'))
            value = apply(op, value, rhs)
        return value
    result = parse()
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Chapter 3 worked solutions (continued)

```
    if position != len(tokens): raise ValueError('reference leftover')
    return result

def main():
    cases = ['9-4', '(9-4)*(2+3)', '9-4*2/1+3', '2^3^2', '9-4-2']
    assert postfix('9-4')[0] == ['9','4','-']
    assert postfix('9-4*2/1+3')[0] == ['9','4','2','*','1','/','-','3','+']
    assert postfix('2^3^2')[0] == ['2','3','2','^','^']
    legacy_output_before_eof = postfix('9-4')[1][-2][2]
    assert legacy_output_before_eof == '9 4'
    invalid = ['(9-4','9-4)','9-','9 4','()','9+(4*)','9**4', '', '(9)(4)']
    for expression in invalid:
        try: postfix(expression)
        except ValueError: pass
        else: raise AssertionError(expression)
    rng = random.Random(20261024)
    def expression(depth):
        if depth == 0 or rng.randrange(3) == 0: return str(rng.randrange(1,10))
        return '('+expression(depth-1)+rng.choice('+-*')+expression(depth-1)+')'
    generated = [expression(5) for _ in range(500)]
    for value in cases+generated:
        assert evaluate_postfix(postfix(value)[0]) == parse_reference(value)
    print(json.dumps({'legacy_omitted_EOF_output':legacy_output_before_eof,
                      'corrected_postfix_9_minus_4':' '.join(postfix('9-4')[0]),
                      'valid_reference_comparisons':len(cases+generated),
                      'invalid_inputs_rejected':len(invalid),
                      'trace':{case:postfix(case)[1] for case in
cases}},ensure_ascii=False,indent=2))

if __name__ == '__main__':
    main()
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

References and attribution

Ellis Horowitz, Sartaj Sahni, Dinesh Mehta. Fundamentals of Data Structures in C++, 2nd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study